

INFORME

Evaluación práctica de la Unidad 1: Reconociendo lo aprendido

ET0155 - Fundamentos de Big Data

Grupo N.º 8

Miembros del equipo

#	Nombre completo
1	Jonathan David Pasuy Gonzalez

Caso de Estudio: Empresa “Gaseosas Poderosas”

Este trabajo desarrolla, sobre un caso hipotético, todo el ciclo de un proceso ETL de Big Data: se parte de información dispersa en hojas de cálculo y bases de datos independientes por sucursal, se diseña un modelo relacional que las integre, se migra y limpia la información de ventas, y finalmente se consulta y visualiza para apoyar decisiones de negocio.

1.- Descripción de la Tarea

La tarea consistió en montar de principio a fin un pequeño proyecto de Big Data para la empresa ficticia “Gaseosas Poderosas”. El punto de partida fueron los archivos que entregó el docente: el script de creación de la base de datos, un dump con 10.000 operaciones de venta, la hoja de cálculo del DANE con departamentos y municipios, y dos algoritmos en Python para cargar la información.

Lo primero fue crear la base de datos en PostgreSQL y correr el algoritmo ETL original, que lee la hoja de cálculo del DANE y construye las tablas de departamentos y municipios generando códigos únicos (el famoso problema de que dos códigos DANE distintos, al pasarlos a entero, podían chocar). Sobre esa base ya cargada se trabajó en tres frentes: (1) agregar el nivel de “región” que pedía el punto 5, creando la tabla regiones y modificando el algoritmo ETL para que valore el campo `id_region` en operaciones; (2) hacer el análisis exploratorio de la tabla operaciones para encontrar los registros con problemas (fechas mal escritas, cantidades en cero o negativas, departamentos o productos sin código) y corregirlos uno por uno dejando trazabilidad de qué se cambió y por qué; y (3) construir las consultas SQL pedidas sobre la vista `vista_operaciones` y armar los gráficos de Pareto y de torta en Excel.

Todo el trabajo de base de datos y limpieza se hizo con PostgreSQL, y la carga y transformación con Python (psycopg2 y pandas). Los resultados de cada paso quedan documentados en las secciones siguientes.

2.- Diagrama de Entidad-Relación (Diagrama de Chen)

El diagrama incluye las 5 entidades del modelo (no se incluye la tabla “tamaño” porque es solo para las pruebas de rendimiento del punto 10). Se muestran las relaciones 1:M entre región-departamento, departamento-municipio, y las tres relaciones 1:M que tiene operaciones con departamento, municipio y producto. También se dejó marcada (línea punteada) la relación directa región-operaciones, que es el campo id_region que se agregó para no tener que pasar siempre por departamento en las consultas por región.

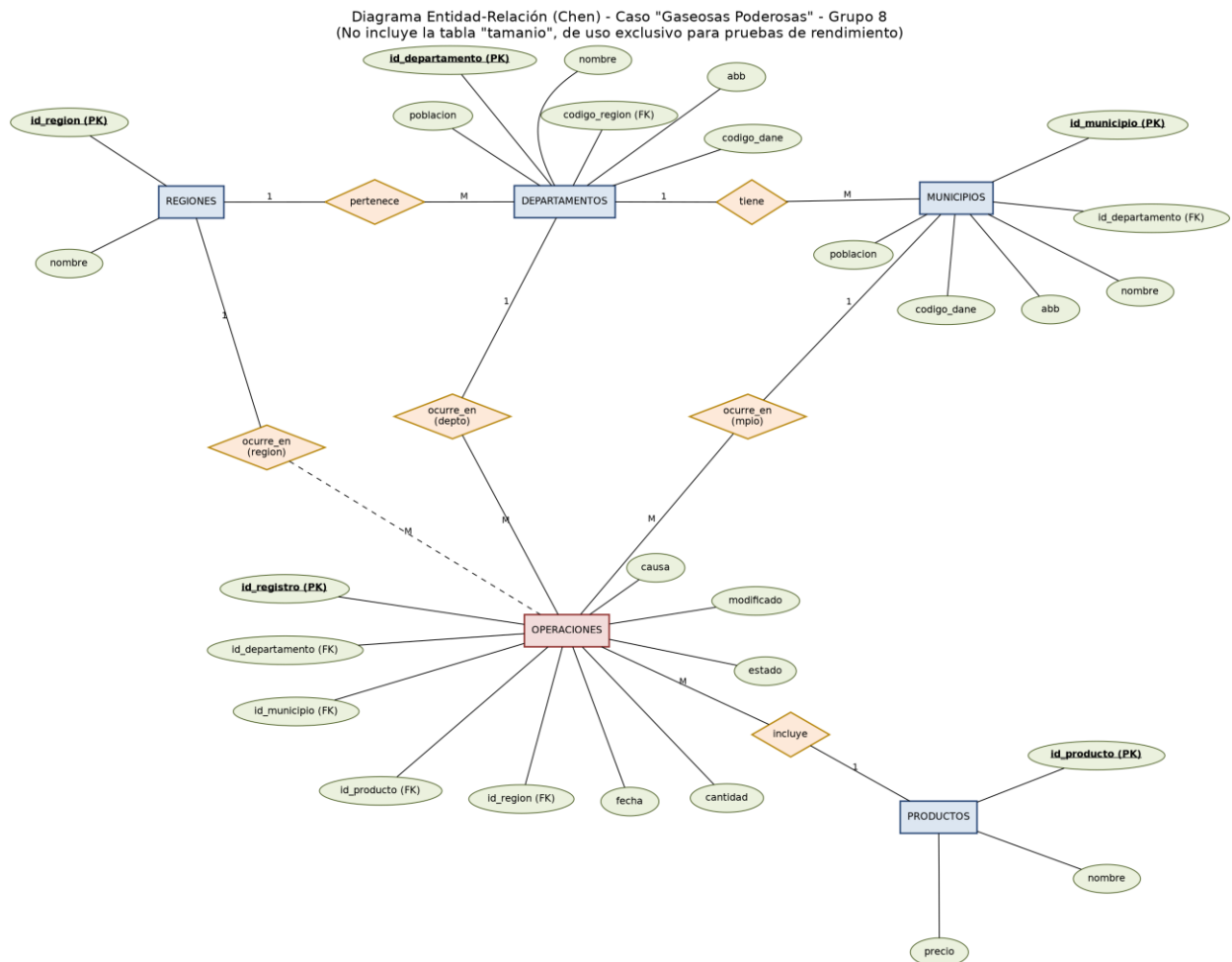


Figura 1. Diagrama Entidad-Relación de Chen del caso “Gaseosas Poderosas”

3.- Diccionario de Datos

A continuación se detalla cada tabla de la base de datos “bigdata”, incluyendo la tabla regiones y el campo id_region que se agregaron para resolver el punto 5.

Tabla: regiones

Campo	Tipo	Tamaño	Nulo	Clave	Descripción
id_region	integer	-	NOT NULL	PK	Código de la región (1 a 6)
nombre	varchar	50	NOT NULL	-	Nombre de la región

Tabla: departamentos

Campo	Tipo	Tamaño	Nulo	Clave	Descripción
id_departamento	integer	-	NOT NULL	PK	57 (Colombia) + código DANE de 2 dígitos
nombre	varchar	70	NOT NULL	-	Nombre del departamento
abb	varchar	3	-	-	Abreviatura (no se usa en esta tarea)
codigo_dane	varchar	10	-	UK	Código DANE original del departamento
codigo_region	integer	-	-	FK -> regiones.id_region	Región a la que pertenece
poblacion	integer	-	-	-	Población (no se usa en esta tarea)

Tabla: municipios

Campo	Tipo	Tamaño	Nulo	Clave	Descripción
id_municipio	integer	-	NOT NULL	PK	id_departamento + consecutivo de 3 dígitos
id_departamento	integer	-	NOT NULL	FK -> departamentos.id_departamento	Departamento al que pertenece
nombre	varchar	70	NOT NULL	-	Nombre del municipio
abb	varchar	5	-	-	Abreviatura (no se usa en esta tarea)
codigo_dane	varchar	10	-	UK	Código DANE original del municipio
poblacion	integer	-	-	-	Población (no se usa en esta tarea)

Tabla: productos

Campo	Tipo	Tamaño	Nulo	Clave	Descripción
id_producto	integer	-	NOT NULL	PK	Código del producto (1 a 4)
nombre	varchar	20	NOT NULL	UK	Nombre de la gaseosa
precio	integer	-	-	-	Precio unitario

Tabla: operaciones

Campo	Tipo	Tamaño	Nulo	Clave	Descripción
id_registro	integer	-	NOT NULL	PK	Identificador único de la venta
id_departamento	integer	-	NOT NULL	FK -> departamentos	Departamento de la venta
id_municipio	integer	-	NOT NULL	FK -> municipios	Municipio de la venta
id_producto	integer	-	NOT NULL	FK -> productos	Producto vendido
id_region	integer	-	-	FK -> regiones	Región (agregado por el equipo, punto 5)
fecha	varchar	10	-	-	Fecha de la venta, formato AAAA-MM-DD
cantidad	integer	-	-	-	Unidades vendidas
estado	varchar	1	-	-	Estado del registro (heredado del dump original)
modificado	varchar	1	-	-	'S' si el registro fue corregido en la limpieza, 'N' si ya era válido (agregado por el equipo)
causa	text	-	-	-	Detalle de qué se corrigió y por qué (agregado por el equipo)

4.- Corrida del algoritmo ETL

El algoritmo algoritmo-etl-G8.py hace tres cosas: primero lee, fila por fila, la hoja colombia-dane-departamentos.csv y la vuelca en una tabla temporal; luego agrupa esa tabla temporal por departamento y construye el código único de cada uno (57, el prefijo de Colombia, más el código DANE de 2 dígitos); y por último hace lo mismo para cada municipio (código de departamento + un consecutivo de 3 dígitos). Con eso quedan cargadas las 32 filas de departamentos y las 1.121 de municipios que trae la fuente del DANE.

```
bash - algoritmo-etl-G8.py

$ python3 algoritmo-etl-G8.py
PostgreSQL Informacion del Servidor
Python version: 3.12.3
Estas conectado a - PostgreSQL 16.15 (Ubuntu)
Base de datos: bigdata

Tabla Temporal cargada ....
Inicia la carga de informacion ...
DEPARTAMENTOS
02 5702 Antioquia 02 1 ...
... (32 departamentos cargados)
MUNICIPIOS
Carga de munipios: contador 1 ... Medellin
... (1.121 municipios cargados)
Conexion PostgreSQL cerrada

REGION EN OPERACIONES
Campo id_region valorizado en 'operaciones'. Filas afectadas: 10000
Fin del proceso ETL
```

Figura 2. Corrida del algoritmo ETL: carga de departamentos y municipios

```
bash - verificacion de carga

$ psql -d bigdata -c "SELECT count(*) FROM departamentos;"
count
-----
32

$ psql -d bigdata -c "SELECT count(*) FROM municipios;"
count
-----
1121

$ psql -d bigdata -c "SELECT count(*) FROM operaciones;"
count
-----
10000
```

Figura 3. Verificación de los conteos cargados (32 departamentos, 1.121 municipios, 10.000 operaciones)

5.- Modificar el algoritmo ETL para incluir dato de región

Se creó la tabla regiones (con las 6 regiones que ya traía la hoja de cálculo del DANE) y se agregó la columna id_region a la tabla operaciones. Para valorizarla se agregó al final del algoritmo ETL la función valorizarIdRegion(), que actualiza operaciones usando el codigo_region que cada departamento ya tenía cargado:

```
def valorizarIdRegion(conn, cursor):
    comando_sql = '''UPDATE operaciones o
                      SET id_region = d.codigo_region
                      FROM departamentos d
                      WHERE d.id_departamento = o.id_departamento;'''
    cursor.execute(comando_sql)
    conn.commit()
```

Con esto, cada una de las 10.000 operaciones quedó con su región asignada automáticamente (heredada del departamento), sin tener que volver a tocar la hoja de cálculo original.

6.- Detectar registros con problemas. Transformación de datos

Después de cargar las 10.000 operaciones se hizo un análisis exploratorio con SQL y se encontraron exactamente 30 registros con problemas (0,3% del total), repartidos en los 5 tipos que planteaba la tarea:

- Tipo A (10 registros): la fecha no viene en formato AAAA-MM-DD. Se revisó cada una a mano, mirando cuál de los tres bloques del valor no podía ser un mes (por ser mayor a 12) para saber si era el día, y así reconstruir la fecha real.
- Tipo B (5 registros): la cantidad viene en 0. Se imputó con el promedio de cantidad vendida en ese mismo municipio, usando solo los registros con cantidad mayor a 0 para no ensuciar el promedio.
- Tipo C (5 registros): la cantidad viene negativa. Se asumió que es un error de digitación (el operador puso el signo de más) y se corrigió tomando el valor absoluto.
- Tipo D (5 registros): falta el código de departamento (viene en 0). Como el id_municipio se construye como id_departamento + consecutivo, el departamento se recupera tomando los primeros dígitos del id_municipio.
- Tipo E (5 registros): falta el código de producto (viene en 0), y en todos los casos el municipio es Tamesis, Antioquia, donde -según el caso de estudio- solo se vende NARANJITA. Se asignó ese código directamente.

Se agregaron dos columnas nuevas a operaciones: modificado ('S'/'N') y causa (texto con el detalle de qué se corrigió). Los 9.970 registros restantes quedaron marcados como 'N' (válidos, sin cambios). El script completo con los UPDATE de cada corrección es script-limpieza-transformacion.sql, entregado junto con este informe.

```
bash - limpieza y transformacion

$ psql -d bigdata -f script-limpieza-transformacion.sql
UPDATE 1 -- (x10, Tipo A: fechas)
UPDATE 5 -- Tipo B: cantidades en 0, imputadas con promedio de municipio
UPDATE 5 -- Tipo C: cantidades negativas, se corrige el signo
UPDATE 5 -- Tipo D: id_departamento en 0, recuperado desde id_municipio
UPDATE 9970 -- registros validos marcados como 'N'
UPDATE 10000 -- id_region valorizado en toda la tabla

$ psql -d bigdata -c "SELECT (SELECT count(*) FROM operaciones) AS total_operaciones,
                          (SELECT count(*) FROM vista_operaciones) AS total_vista;"
total_operaciones | total_vista
-----+-----
10000 | 10000
(la vista coincide 100% con la tabla: la limpieza quedo completa)
```

Figura 4. Corrida del script de limpieza: 30 registros corregidos, y verificación de que vista_operaciones coincide 100% con operaciones

Cuadro de registros con problemas:

id_registro	Campo (BD)	Tipo	Problema detectado	Solución aplicada
10001	fecha	Tipo A	Fecha "2024-8-21" sin cero en el mes	Se normaliza a 2024-08-21
10014	cantidad	Tipo C	Cantidad negativa (-198)	Se corrige el signo -> 198
10225	cantidad	Tipo B	Cantidad en 0	Se imputa el promedio del municipio -> 136
10326	id_departamento	Tipo D	Código de departamento en 0	Se recupera desde id_municipio -> 5715

id_registro	Campo (BD)	Tipo	Problema detectado	Solución aplicada
10686	id_producto	Tipo E	Código de producto en 0 en Tamesis	Único producto vendido allí es NARANJITA -> 4
11599	fecha	Tipo A	Fecha "24-02-08" con año de 2 dígitos	Se completa el siglo -> 2024-02-08
11734	id_producto	Tipo E	Código de producto en 0 en Tamesis	Se asigna NARANJITA -> 4
11907	cantidad	Tipo C	Cantidad negativa (-144)	Se corrige el signo -> 144
12110	fecha	Tipo A	Fecha "024-07-17" con año incompleto	Se corrige a 2024-07-17
12188	id_departamento	Tipo D	Código de departamento en 0	Se recupera desde id_municipio -> 5715
13161	cantidad	Tipo B	Cantidad en 0	Se imputa el promedio del municipio -> 105
13945	fecha	Tipo A	Fecha "13-12-2024" en formato DD-MM-AAAA	Se reordena a 2024-12-13
14020	id_departamento	Tipo D	Código de departamento en 0	Se recupera desde id_municipio -> 5723
14368	fecha	Tipo A	Fecha "24-28-10" (28 no puede ser mes)	Formato AA-DD-MM -> 2024-10-28
15208	cantidad	Tipo C	Cantidad negativa (-175)	Se corrige el signo -> 175
15412	cantidad	Tipo B	Cantidad en 0	Se imputa el promedio del municipio -> 79
15955	fecha	Tipo A	Fecha "24-27-09" (27 no puede ser mes)	Formato AA-DD-MM -> 2024-09-27
16558	fecha	Tipo A	Fecha "01-02-24" en formato DD-MM-AA	Se completa a 2024-02-01
16635	id_departamento	Tipo D	Código de departamento en 0	Se recupera desde id_municipio -> 5741
17125	fecha	Tipo A	Fecha "12-12-2024" en formato DD-MM-AAAA	Se reordena a 2024-12-12
17341	id_producto	Tipo E	Código de producto en 0 en Tamesis	Se asigna NARANJITA -> 4
17528	cantidad	Tipo B	Cantidad en 0	Se imputa el promedio del municipio -> 79
18056	fecha	Tipo A	Fecha "24-09-09" con año de 2 dígitos	Se completa el siglo -> 2024-09-09
18126	cantidad	Tipo C	Cantidad negativa (-181)	Se corrige el signo -> 181
18469	id_departamento	Tipo D	Código de departamento en 0	Se recupera desde id_municipio -> 5763

id_registro	Campo (BD)	Tipo	Problema detectado	Solución aplicada
18911	id_producto	Tipo E	Código de producto en 0 en Tamesis	Se asigna NARANJITA -> 4
19181	cantidad	Tipo B	Cantidad en 0	Se imputa el promedio del municipio -> 98
19195	id_producto	Tipo E	Código de producto en 0 en Tamesis	Se asigna NARANJITA -> 4
19538	cantidad	Tipo C	Cantidad negativa (-112)	Se corrige el signo -> 112
19680	fecha	Tipo A	Fecha "11-23-2024" (23 no puede ser mes)	Formato MM-DD-AAAA -> 2024-11-23

7.- Tabla de consultas SQL

Todas las consultas se construyeron sobre la VIEW vista_operaciones, tal como exige la tarea, y se verificaron contra la tabla operaciones para confirmar que daban el mismo resultado (ver Figura 4: 10.000 = 10.000).

7.1 Top 8 departamentos por monto de ventas

```
SELECT departamento, SUM(venta) AS monto_total
FROM vista_operaciones
GROUP BY departamento
ORDER BY monto_total DESC
LIMIT 8;
```

7.2 Top 15 municipios de Antioquia por cantidad vendida

```
SELECT municipio, SUM(cantidad) AS cantidad_total
FROM vista_operaciones
WHERE departamento = 'Antioquia'
GROUP BY municipio
ORDER BY cantidad_total DESC
LIMIT 15;
```

7.3 Top 5 departamentos con mayor cantidad de MANZALOCA

```
SELECT departamento, SUM(cantidad) AS cantidad_total
FROM vista_operaciones
WHERE producto = 'MANZALOCA'
GROUP BY departamento
ORDER BY cantidad_total DESC
LIMIT 5;
```

7.4 Los 5 municipios con menor monto de ventas

```
SELECT departamento, municipio, SUM(venta) AS monto_total
FROM vista_operaciones
GROUP BY departamento, municipio
ORDER BY monto_total ASC
LIMIT 5;
```


7.5 Cantidad de gaseosas vendidas por producto y región

```
SELECT r.nombre AS region, vo.producto, SUM(vo.cantidad) AS cantidad_total
FROM vista_operaciones vo
JOIN operaciones op ON op.id_registro = vo.id_registro
JOIN regiones r ON r.id_region = op.id_region
GROUP BY r.nombre, vo.producto
ORDER BY cantidad_total DESC;
```

7.6 Monto de ventas por producto en Antioquia

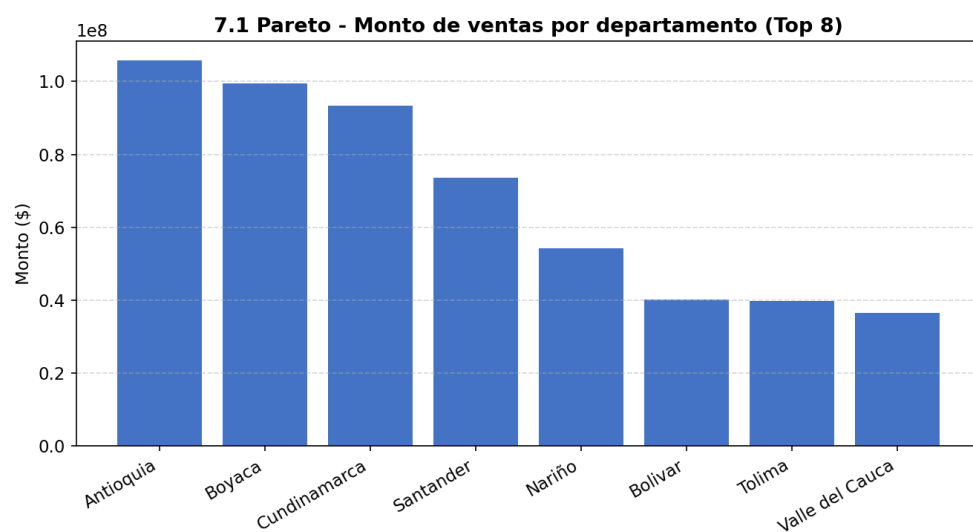
```
SELECT producto, SUM(venta) AS monto_total
FROM vista_operaciones
WHERE departamento = 'Antioquia'
GROUP BY producto
ORDER BY monto_total DESC;
```

Los 6 scripts se entregan también por separado (carpeta consultas/) para subirlos al repositorio Git del equipo.

8.- Gráficos

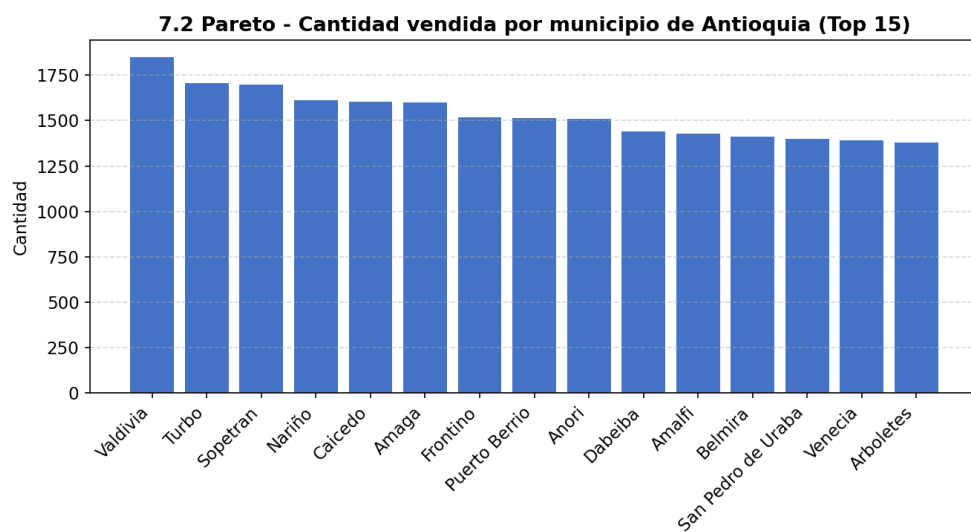
8.1.- Gráfico de Pareto - Consulta 7.1

Departamento	Monto total
Antioquia	105.759.800
Boyaca	99.508.500
Cundinamarca	93.349.300
Santander	73.633.600
Nariño	54.167.600
Bolivar	40.279.200
Tolima	39.817.700
Valle del Cauca	36.605.600



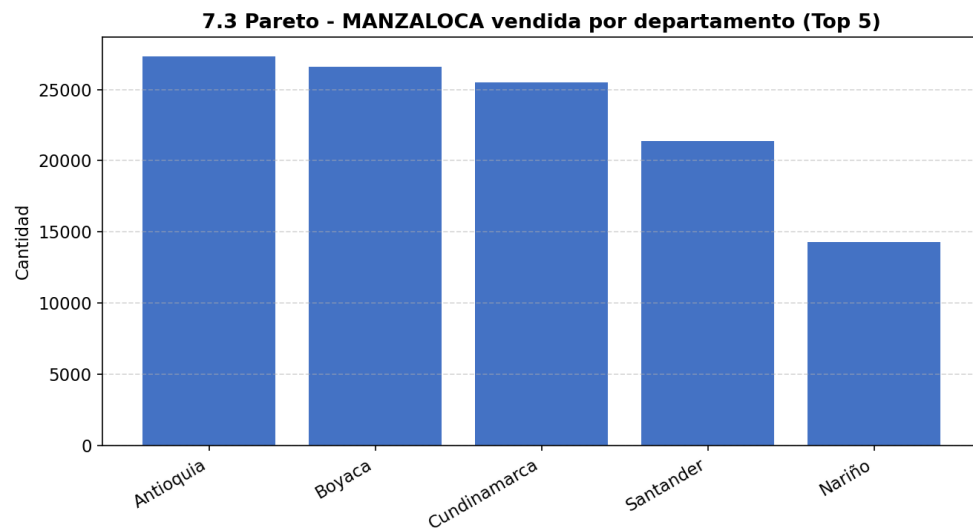
8.2.- Gráfico de Pareto - Consulta 7.2

Municipio	Cantidad total
Valdivia	1.850
Turbo	1.708
Sopetran	1.699
Nariño	1.611
Caicedo	1.604
Amaga	1.600
Frontino	1.520
Puerto Berrio	1.514
Anori	1.510
Dabeiba	1.441
Amalfi	1.430
Belmira	1.412
San Pedro de Uraba	1.399
Venecia	1.391
Arboletes	1.380



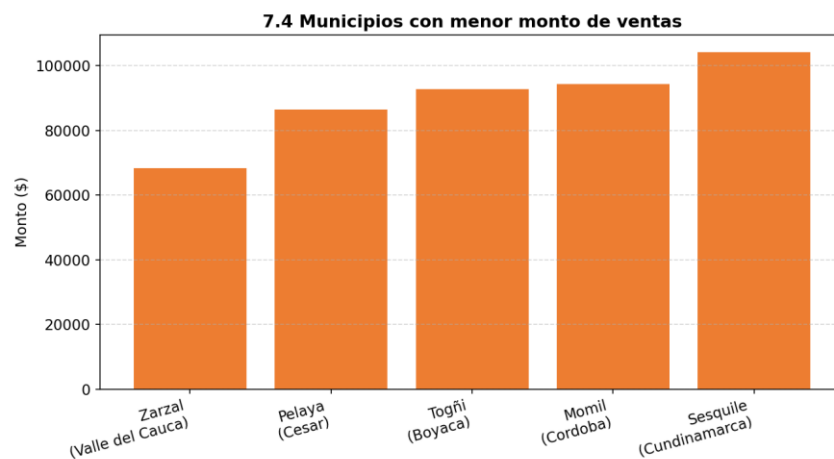
8.3.- Gráfico de Pareto - Consulta 7.3

Departamento	Cantidad total
Antioquia	27.305
Boyaca	26.577
Cundinamarca	25.510
Santander	21.376
Nariño	14.273



8.4.- Gráfico de Pareto - Consulta 7.4

Departamento	Municipio	Monto total
Valle del Cauca	Zarzal	68.200
Cesar	Pelaya	86.400
Boyaca	Togñi	92.800
Cordoba	Momil	94.200
Cundinamarca	Sesquile	104.200

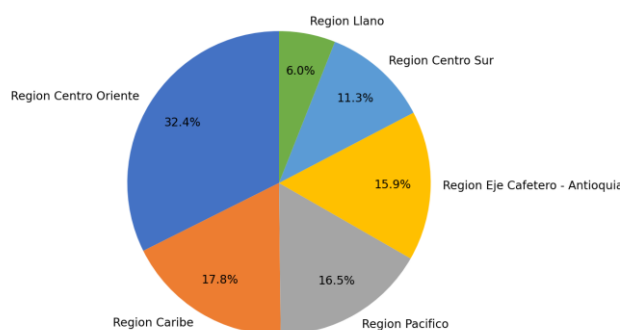


8.5.- Gráfico de Torta - Consulta 7.5

La consulta 7.5 arroja 24 filas (4 productos x 6 regiones); en el archivo Excel adjunto está la tabla completa. Para el gráfico de torta se resumió el total vendido por región:

Región	Cantidad total
Region Centro Oriente	326.259
Region Caribe	179.402
Region Pacifico	166.663
Region Eje Cafetero - Antioquia	160.691
Region Centro Sur	114.208
Region Llano	60.523

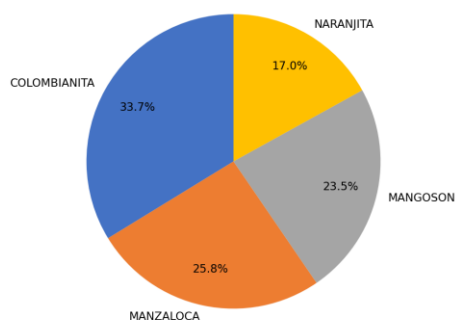
7.5 Torta - Cantidad de gaseosas vendidas por region



8.6.- Gráfico de Torta - Consulta 7.6

Producto	Monto total
COLOMBIANITA	35.680.800
MANZALOCA	27.305.000
MANGOSON	24.820.200
NARANJITA	17.953.800

7.6 Torta - Monto de ventas por producto en Antioquia



El archivo bigdata-et0155-tarea-ETL-v2-graficos-equipo_8.xlsx (entregado junto con este informe) tiene cada consulta en su propia pestaña, con la tabla de resultados y su gráfico correspondiente, tal como se pedía en los requerimientos en el Moodle.

10.- Determinar el tiempo y tamaño de procesamiento de un conjunto de registros

Se modificó algoritmo-calculo-tamano.py para medir el tiempo de procesamiento con time.time() y para que la consulta de tamaño apunte a la base de datos correcta (“bigdata” en vez de “bigdata-g050”, que era de otro grupo). También se cambió la manera de elegir el municipio al azar: en vez de una consulta SELECT ... ORDER BY RANDOM() por cada fila (muy costosa), se trae la lista de municipios una sola vez a memoria.

```
bash - algoritmo-calculo-tamano-G8.py

$ python3 algoritmo-calculo-tamano-G8.py 10000
Registros generados: 10000
Tiempo de procesamiento: 459.04 milisegundos
Tamano de la base de datos: 11 MB
Tamano de la tabla 'tamano': 696 kB
Porcentaje de 'tamano' sobre el total: 5.98 %

$ python3 algoritmo-calculo-tamano-G8.py 100000
Registros generados: 100000
Tiempo de procesamiento: 4477.42 milisegundos
Tamano de la base de datos: 17 MB
Tamano de la tabla 'tamano': 6696 kB
Porcentaje de 'tamano' sobre el total: 37.98 %

$ python3 algoritmo-calculo-tamano-G8.py 1000000
Registros generados: 1000000
Tiempo de procesamiento: 43254.07 milisegundos
Tamano de la base de datos: 76 MB
Tamano de la tabla 'tamano': 65 MB
Porcentaje de 'tamano' sobre el total: 85.92 %
```

Figura 5. Corridas del algoritmo de tamaño para 10.000, 100.000 y 1.000.000 de registros

```
bash - algoritmo-calculo-tamano-G8.py 10000000

$ python3 algoritmo-calculo-tamano-G8.py 10000000
Registros generados: 10000000
Tiempo de procesamiento: 449421.48 milisegundos (~7.5 minutos)
Tamano de la base de datos: 662 MB
Tamano de la tabla 'tamano': 651 MB
Porcentaje de 'tamano' sobre el total: 98.38 %
Conexion PostgreSQL cerrada
```

Figura 5b. Corrida del algoritmo de tamaño para 10.000.000 de registros (~7,5 minutos)

Cantidad de registros	Tiempo de procesamiento (ms)	Tamaño tabla "tamano"	Tamaño BD "bigdata"	% de "tamano" sobre el total
10.000	459	696 kB	11 MB	5,98 %
100.000	4.477	6.696 kB	17 MB	37,98 %
1.000.000	43.254	65 MB	76 MB	85,92 %
10.000.000	449.421	651 MB	662 MB	98,38 %

11.- Análisis de los resultados

Antioquia es el departamento que más factura (\$105,8 millones), pero por poco: Boyacá lo sigue muy de cerca con \$99,5 millones, y ambos están bastante por encima del resto. En cantidad de MANZALOCA vendida pasa algo parecido, Antioquia y Boyacá casi empatados arriba de los demás, lo que sugiere que en esos dos departamentos hay simplemente más puntos de venta activos o mercados más grandes, no que un producto en particular esté “de moda” ahí.

Llama la atención que dentro de Antioquia, que es el departamento líder, los municipios con más ventas (Valdivia, Turbo, Sopetrán) no son los más grandes o conocidos del departamento (no aparece Medellín en el top, por ejemplo). Eso puede ser un reflejo de que los datos son generados de forma aleatoria para la tarea y no de un patrón real de mercado, pero en un caso real ameritaría revisar si falta capturar ventas de los municipios grandes o si de verdad el producto está mejor posicionado en municipios intermedios.

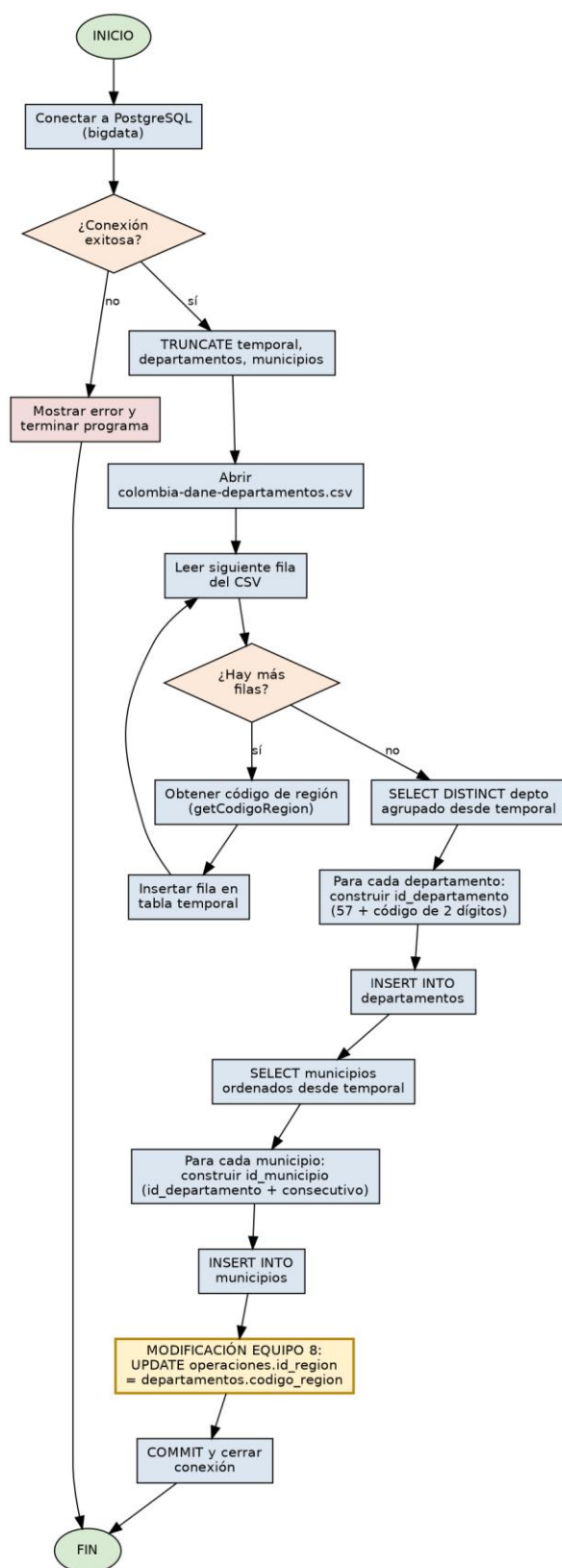
Por producto, COLOMBIANITA es la que más factura en Antioquia (y muy probablemente en el resto del país, a juzgar por su precio más alto: \$1.200 contra \$600-\$1.000 de las demás), aunque no es la que más se vende en cantidad de unidades. Esto es un punto interesante para la gerencia: el producto “estrella” en plata no es el que más veces se compra, así que antes de decidir en qué producto invertir en mercadeo conviene mirar las dos métricas por separado.

Por región, Centro Oriente concentra casi un tercio de todas las unidades vendidas (32,4%), muy por encima de las demás regiones, mientras que Llano apenas llega al 6%. Tiene sentido pedir explicación de esa diferencia: ¿es que Centro Oriente tiene más municipios o más población, o es que ahí la marca está mejor distribuida? Con la información que se tiene no se puede saber con certeza, pero es exactamente el tipo de pregunta que un analista de datos debería llevarle a la gerencia junto con el dato.

En cuanto a los municipios con menor venta, se ve que están repartidos en departamentos distintos (Valle del Cauca, Cesar, Boyacá, Córdoba, Cundinamarca), es decir que no es un problema de una sola zona del país, sino más bien de municipios pequeños o poco atendidos dentro de departamentos que en general venden bien. Ahí habría una oportunidad de negocio: reforzar la distribución en esos municipios puntuales en vez de asumir que todo el departamento necesita atención.

12.- Diagrama de flujo del programa Python de procesamiento ETL

Diagrama de flujo de algoritmo-etl-G8.py, incluyendo la modificación del equipo (recuadro amarillo) que valoriza el campo id_region al final del proceso.



13.- Conclusiones INDIVIDUALES

Jonathan David Pasuy Gonzalez

Esta tarea me sirvió para entender el proceso ETL más allá de la teoría: una cosa es saber que “primero se extrae, luego se transforma y luego se carga”, y otra muy distinta es toparse con datos reales que vienen con errores (fechas mal escritas, cantidades negativas, códigos en cero) y tener que decidir, caso por caso, cómo recuperarlos sin perder información. Me quedó claro que la parte de limpieza de datos es la que más tiempo consume en un proyecto de este tipo, y que documentar bien cada corrección (con el campo causa, por ejemplo) es tan importante como hacer la corrección misma, porque permite auditar después qué se tocó y por qué. También fue un buen ejercicio combinar Python y SQL: usar Python para automatizar la carga y SQL para el análisis exploratorio y las consultas de negocio, que es como se trabaja en la práctica.

14.- Video explicativo del desarrollo completo de la tarea

Link del video:

Link del repositorio: